

Arrays

So let us let our ordinals start at zero: an element's ordinal (subscript) equals the number of elements preceding it in the sequence. And the moral of the story is that we had better regard — after all those centuries! — zero as a most natural number.

— Edsger W. Dijkstra

EWD831: Why Numbering Should Start at Zero, 1982

Programs rely on data structures to store and manipulate data in complex ways. Therefore, verifying these programs requires decision procedures that reason about the data structures. This chapter discusses fragments and decision procedures for reasoning about arrays and array-like data structures: arrays with anonymous or uninterpreted indices (Section 11.1), the familiar arrays of C and other imperative languages with integer indices (Section 11.2), and hashables (Section 11.3).

Unlike the quantifier-free fragment of arrays presented in Chapter 9, the fragments of this chapter allow some quantification. Specifically, at most one **quantifier alternation** is allowed: for satisfiability, formulae may have universal quantifiers in addition to the implicit existential ones. The technique that underlies the decision procedures of this chapter is **quantifier instantiation**. A satisfiability decision procedure based on quantifier instantiation identifies a finite set of terms suggested by the given formula F ; it then replaces universal quantification with finite conjunction over the set of terms. The resulting formula F' is quantifier-free and equisatisfiable to the given one. Finally, it applies to F' a satisfiability decision procedure for the quantifier-free fragment (for example, a Nelson-Oppen combination of procedures; see Chapter 10); the answer applies also to F .

Section 11.1 studies the **array property fragment** of T_A in which one (restricted) quantifier alternation is allowed. Section 11.2 examines the familiar case in which arrays are indexed by integers. Reasoning about integer indices allows making relative comparisons via $\leq$, providing more power than the case in which indices are only compared via equality.

Hashtables are another important data type. They are similar to arrays with uninterpreted indices in that their indices, or keys, can only be compared via equality. However, hashables allow two new interesting operations: first, a key/value pair can be removed; and second, a hashtable's domain — its set of keys that it maps to values — can be read. Section 11.3 formalizes reasoning about hashables in the theory T_H and then presents a decision procedure for the **hashtable property fragment** of T_H . The procedure operates by transforming Σ_H -formulae to Σ_A -formulae in the array property fragment such that the original formula is T_H -satisfiable iff the constructed formula is T_A -satisfiable.

11.1 Arrays with Uninterpreted Indices

The quantifier-free fragment of T_A enables basic reasoning in the presence of arrays. For verification purposes, it allows verifying properties of individual elements but not of entire arrays. However, in practice, it is useful to be able to reason about properties such as equality between arrays. Using combinations of theories (see Chapter 10), one would also like to reason about properties such as that all integer elements of an array are positive.

11.1.1 Array Property Fragment

In this section, we define a decidable fragment of T_A that allows some quantification. This fragment is called the **array property fragment** because it allows specifying basic properties of arrays, not just properties of array elements. The principal characteristic of the array property fragment is that array indices can be universally quantified with some restrictions.

Example 11.1. In the Σ_A -formula

$$\forall j. a\langle i \triangleleft v \rangle[j] = a[j] \wedge a[i] \neq v ,$$

the first conjunct asserts that $a\langle i \triangleleft v \rangle$ and a are equal. This formula is T_A -unsatisfiable. ■

Unfortunately, the use of universal quantification must be restricted to avoid undecidability (see Section 11.4 for further discussion). An **array property** is a Σ_A -formula of the form

$$\forall \bar{i}. F[\bar{i}] \rightarrow G[\bar{i}]$$

in which $\bar{i}$ is a list of variables, and $F[\bar{i}]$ and $G[\bar{i}]$ are the **index guard** and the **value constraint**, respectively. The index guard $F[\bar{i}]$ is any Σ_A -formula that is syntactically constructed according to the following grammar:

$$\begin{aligned}
\text{iguard} &\rightarrow \text{iguard} \wedge \text{iguard} \mid \text{iguard} \vee \text{iguard} \mid \text{atom} \\
\text{atom} &\rightarrow \text{var} = \text{var} \mid \text{evar} \neq \text{var} \mid \text{var} \neq \text{evar} \mid \top \\
\text{var} &\rightarrow \text{evar} \mid \text{uvar}
\end{aligned}$$

where *uvar* is any universally quantified index variable, and *evar* is any constant or unquantified (that is, implicitly existentially quantified) variable.

Additionally, a universally quantified index can occur in a value constraint $G[\bar{z}]$ only in a read $a[i]$, where a is an array term. The read cannot be nested; for example, $a[b[i]]$ is not allowed.

The array property fragment of T_A then consists of formulae that are Boolean combinations of quantifier-free Σ_A -formulae and array properties.

Example 11.2. The antecedent of the implication in the Σ_A -formula

$$F : \forall i. i \neq a[k] \rightarrow a[i] = a[k]$$

is not a legal index guard since $a[k]$ is not a variable (neither a *uvar* nor an *evar*); however, a simple manipulation makes it conform:

$$F' : v = a[k] \wedge \forall i. i \neq v \rightarrow a[i] = a[k]$$

Here, $i \neq v$ is a legal index guard, and $a[i] = a[k]$ is a legal value constraint. F and F' are equisatisfiable.

However, no amount of manipulation can make the following formula conform:

$$G : \forall i. i \neq a[i] \rightarrow a[i] = a[k] .$$

Thus, G is not in the array property fragment. ■

Example 11.3. The array property fragment allows expressing equality between arrays, a property referred to as **extensionality**: two arrays are equal precisely when their corresponding elements are equal. For given formula

$$F : \dots \wedge a = b \wedge \dots$$

with array terms a and b , rewrite F as

$$F' : \dots \wedge (\forall i. a[i] = b[i]) \wedge \dots .$$

F and F' are equisatisfiable. Moreover, the index guard in the literal is just $\top$, and the value constraint $a[i] = b[i]$ obeys the requirement that i appear only as an index in read terms.

Recall that the theory of arrays with extensionality T_A^- (see Section 3.6) augments T_A with the following axiom:

$$\forall a, b. (\forall i. a[i] = b[i]) \leftrightarrow a = b \quad (\text{extensionality})$$

The universal quantifier of the array property fragment allows expressing equality between arrays directly.

Subsequently, where convenient, we write equality $a = b$ between arrays to abbreviate $\forall i. a[i] = b[i]$. ■

Example 11.4. Reasoning about arrays is most useful when we can say something interesting about their elements. Suppose array elements are interpreted in some theory T with signature Σ . Then we can assert that all elements of an array have some property $F[x]$, where F is a quantifier-free Σ -formula: $\forall i. F[a[i]]$; or that all but a finite number of elements have some property $F[x]$:

$$\forall i. \left(\bigwedge_{k=1}^n i \neq j_k \right) \rightarrow F[a[i]] .$$

■

11.1.2 Decision Procedure

The idea of the decision procedure for the array property fragment is to reduce universal quantification to finite conjunction. It constructs a finite set of index terms such that examining only these positions of the arrays is sufficient to decide satisfiability.

Example 11.5. Consider the formula

$$F : a\langle i \triangleleft v \rangle = a \wedge a[i] \neq v ,$$

which expands to

$$F' : \forall j. a\langle i \triangleleft v \rangle[j] = a[j] \wedge a[i] \neq v .$$

Intuitively, to determine that F' is T_A -unsatisfiable requires merely examining index i :

$$F'' : \left(\bigwedge_{j \in \{i\}} a\langle i \triangleleft v \rangle[j] = a[j] \right) \wedge a[i] \neq v ,$$

or simply

$$a\langle i \triangleleft v \rangle[i] = a[i] \wedge a[i] \neq v .$$

Simplifying,

$$v = a[i] \wedge a[i] \neq v ,$$

it is clear that this formula, and thus F , is T_A -unsatisfiable. ■

Given array property formula F , decide its T_A -satisfiability as follows.

Step 1

Put F in NNF.

Step 2

Apply the following rule exhaustively to remove writes:

$$\frac{F[a\langle i \triangleleft v \rangle]}{F[a'] \wedge a'[i] = v \wedge (\forall j. j \neq i \rightarrow a[j] = a'[j])} \text{ for fresh } a' \quad (\text{write})$$

Rules should be read from top to bottom. For example, this rule states that given a formula F containing an occurrence of a write term $a\langle i \triangleleft v \rangle$, substitute every occurrence of $a\langle i \triangleleft v \rangle$ with a fresh variable a' and conjoin several new conjuncts.

This step deconstructs write terms in a straightforward manner, essentially encoding the (read-over-write) axioms into the new formula. After an application of the rule, the resulting formula contains at least one fewer write terms than the given formula.

Step 3

Apply the following rule exhaustively to remove existential quantification:

$$\frac{F[\exists \bar{i}. G[\bar{i}]]}{F[G[\bar{j}]]} \text{ for fresh } \bar{j} \quad (\text{exists})$$

Existential quantification can arise during Step 1 if the given formula has a negated array property.

Step 4

Steps 4-6 accomplish the reduction of universal quantification to finite conjunction. The main idea is to select a set of symbolic index terms on which to instantiate all universal quantifiers. The proof of Theorem 11.7 argues that the following set is sufficient for correctness.

From the output F_3 of Step 3, construct the **index set** $\mathcal{I}$:

$$\begin{aligned} \mathcal{I} = & \cup \{ \lambda \} \\ & \cup \{ t : \cdot[t] \in F_3 \text{ such that } t \text{ is not a universally quantified variable} \} \\ & \cup \{ t : t \text{ occurs as an } \textit{eval} \text{ in the parsing of index guards} \} \end{aligned}$$

Recall that *eval* is any constant or unquantified variable. This index set is the finite set of indices that need to be examined. It includes all terms t that occur in some read $a[t]$ anywhere in F (unless it is a universally quantified variable) and all terms t that are compared to a universally quantified variable in some index guard. λ is a fresh constant that represents all other index positions that are not explicitly in $\mathcal{I}$.

Step 5

Apply the following rule exhaustively to remove universal quantification:

$$\frac{H[\forall \vec{i}. F[\vec{i}] \rightarrow G[\vec{i}]]}{H \left[\bigwedge_{\vec{i} \in \mathcal{I}^n} (F[\vec{i}] \rightarrow G[\vec{i}]) \right]} \quad (\text{forall})$$

where n is the size of the list of quantified variables $\vec{i}$. This is the key step. It replaces universal quantification with finite conjunction over the index set. The notation $\vec{i} \in \mathcal{I}^n$ means that the variables $\vec{i}$ range over all n -tuples of terms in $\mathcal{I}$.

Step 6

From the output F_5 of Step 5, construct

$$F_6 : F_5 \wedge \bigwedge_{i \in \mathcal{I} \setminus \{\lambda\}} \lambda \neq i .$$

The new conjuncts assert that the variable λ introduced in Step 4 is indeed unique: it does not equal any other index mentioned in F_5 .

Step 7

Decide the T_A -satisfiability of F_6 using the decision procedure for the quantifier-free fragment.

Suppose array elements are interpreted in some theory T with signature Σ . For deciding the $(T_A \cup T)$ -satisfiability of an array property $(\Sigma_A \cup \Sigma)$ -formula, use a combination decision procedure for the quantifier-free fragment of $T_A \cup T$ in Step 7. Thus, this procedure is a decision procedure precisely when the quantifier-free fragment of $T_A \cup T$ is decidable. Chapter 10 discusses deciding satisfiability in combinations of quantifier-free fragments of theories.

Example 11.6. Consider the array property formula

$$F : a[\ell \triangleleft v][k] = b[k] \wedge b[k] \neq v \wedge a[k] = v \wedge (\forall i. i \neq \ell \rightarrow a[i] = b[i]) .$$

It contains one array property,

$$\forall i. i \neq \ell \rightarrow a[i] = b[i] ,$$

in which the index guard is $i \neq \ell$ and the value constraint is $a[i] = b[i]$. It is already in NNF. According to Step 2, rewrite F as

$$F_2 : a'[k] = b[k] \wedge b[k] \neq v \wedge a[k] = v \wedge (\forall i. i \neq \ell \rightarrow a[i] = b[i]) \\ \wedge a'[\ell] = v \wedge (\forall j. j \neq \ell \rightarrow a[j] = a'[j]) .$$

F_2 does not contain any existential quantifiers. Its index set is

$$\mathcal{I} = \{\lambda\} \cup \{k\} \cup \{\ell\} \\ = \{\lambda, k, \ell\} .$$

According to Step 5, replace universal quantification as follows:

$$F_5 : a'[k] = b[k] \wedge b[k] \neq v \wedge a[k] = v \wedge \bigwedge_{i \in \mathcal{I}} (i \neq \ell \rightarrow a[i] = b[i]) \\ \wedge a'[\ell] = v \wedge \bigwedge_{j \in \mathcal{I}} (j \neq \ell \rightarrow a[j] = a'[j]) .$$

Expanding produces

$$F'_5 : a'[k] = b[k] \wedge b[k] \neq v \wedge a[k] = v \wedge (\lambda \neq \ell \rightarrow a[\lambda] = b[\lambda]) \\ \wedge (k \neq \ell \rightarrow a[k] = b[k]) \wedge (\ell \neq \ell \rightarrow a[\ell] = b[\ell]) \\ \wedge a'[\ell] = v \wedge (\lambda \neq \ell \rightarrow a[\lambda] = a'[\lambda]) \\ \wedge (k \neq \ell \rightarrow a[k] = a'[k]) \wedge (\ell \neq \ell \rightarrow a[\ell] = a'[\ell]) .$$

Simplifying produces

$$F''_5 : a'[k] = b[k] \wedge b[k] \neq v \wedge a[k] = v \wedge (\lambda \neq \ell \rightarrow a[\lambda] = b[\lambda]) \\ \wedge (k \neq \ell \rightarrow a[k] = b[k]) \\ \wedge a'[\ell] = v \wedge (\lambda \neq \ell \rightarrow a[\lambda] = a'[\lambda]) \\ \wedge (k \neq \ell \rightarrow a[k] = a'[k]) .$$

Step 6 distinguishes λ from other members of $\mathcal{I}$:

$$F_6 : a'[k] = b[k] \wedge b[k] \neq v \wedge a[k] = v \wedge (\lambda \neq \ell \rightarrow a[\lambda] = b[\lambda]) \\ \wedge (k \neq \ell \rightarrow a[k] = b[k]) \\ \wedge a'[\ell] = v \wedge (\lambda \neq \ell \rightarrow a[\lambda] = a'[\lambda]) \\ \wedge (k \neq \ell \rightarrow a[k] = a'[k]) \\ \wedge \lambda \neq k \wedge \lambda \neq \ell .$$

Simplifying, we have

$$F'_6 : a'[k] = b[k] \wedge b[k] \neq v \wedge a[k] = v \\ \wedge a[\lambda] = b[\lambda] \wedge (k \neq \ell \rightarrow a[k] = b[k]) \\ \wedge a'[\ell] = v \wedge a[\lambda] = a'[\lambda] \wedge (k \neq \ell \rightarrow a[k] = a'[k]) \\ \wedge \lambda \neq k \wedge \lambda \neq \ell .$$

There are two cases to consider. If $k = \ell$, then $a'[\ell] = v$ and $a'[k] = b[k]$ imply $b[k] = v$, yet $b[k] \neq v$. If $k \neq \ell$, then $a[k] = v$ and $a[k] = b[k]$ imply $b[k] = v$, but again $b[k] \neq v$. Hence, F'_6 is T_A -unsatisfiable, indicating that F is T_A -unsatisfiable.

Verify that the array decision procedure of Section 9.5 reaches the same conclusion for F'_6 . ■

Theorem 11.7 (Sound & Complete). *Consider $(\Sigma_A \cup \Sigma)$ -formula F from the array property fragment of $T_A \cup T$. The output F_6 of Step 6 is $(T_A \cup T)$ -equisatisfiable to F .*

Proof. Inspection proves the equivalence between the input and the output of Steps 1-3. The crux of the proof is that the index set constructed in Step 4 is sufficient for producing a $(T_A \cup T)$ -equisatisfiable quantifier-free formula.

That satisfiability of F implies the satisfiability of F_6 is straightforward: Step 5 weakens universal quantification to finite conjunction. Moreover, the new conjuncts of Step 6 do not affect the satisfiability of F_6 , as λ is a fresh constant.

The opposite direction is more complicated. Assume that $(T_A \cup T)$ -interpretation I is such that $I \models F_6$. We construct a $(T_A \cup T)$ -interpretation J such that $J \models F$.

First, define a projection function that maps Σ_A -terms to terms of the index set $\mathcal{I}$:

$$\text{proj}_I : \Sigma_A\text{-terms} \rightarrow \mathcal{I} .$$

In particular, if $\alpha_I[i] = v_i$ for $i \in \mathcal{I}$ and $\alpha_I[\lambda] = v_\lambda$, then

$$\text{proj}_I(t) = \begin{cases} i & \text{if } \alpha_I[t] = v_i \text{ for some } i \in \mathcal{I} \\ \lambda & \text{otherwise} \end{cases}$$

Extend proj_I to vectors of variables:

$$\text{proj}_I(\vec{i}) = (\text{proj}_I(i_1), \dots, \text{proj}_I(i_n)) .$$

Define J to be like I except for its arrays. Under J , let $a[i] = a[\text{proj}_I(i)]$. Technically, we are specifying how α_J assigns values to terms of F and the array read function $\cdot[\cdot]$; however, we can think in terms of arrays.

To prove that $J \models F$, we focus on a particular subformula $\forall \vec{i}. F[\vec{i}] \rightarrow G[\vec{i}]$. Assume that

$$I \models \bigwedge_{\vec{i} \in \mathcal{I}^n} (F[\vec{i}] \rightarrow G[\vec{i}]) ;$$

then also

$$J \models \bigwedge_{\vec{i} \in \mathcal{I}^n} (F[\vec{i}] \rightarrow G[\vec{i}]) \tag{11.1}$$

by construction of J . We need to prove that

$$J \models \forall \vec{i}. F[\vec{i}] \rightarrow G[\vec{i}] ; \tag{11.2}$$

that is, that

$$J \triangleleft \{\vec{i} \mapsto \vec{v}\} \models F[\vec{i}] \rightarrow G[\vec{i}]$$

for all $\bar{v} \in D_J^n$. Let $K = J \triangleleft \{\bar{i} \mapsto \bar{v}\}$.

To do so, we prove the two implications represented by dashed arrows in the following diagram:

$$\begin{array}{ccc}
 & F[\text{proj}_K(\bar{i})] & \longrightarrow G[\text{proj}_K(\bar{i})] \\
 K \models (1) \uparrow & & \downarrow (2) \\
 & F[\bar{i}] & \xrightarrow{\quad ? \quad} G[\bar{i}]
 \end{array}$$

The top implication holds under K by assumption (11.1). If both implications (1) and (2) hold under K , then the transitivity of implication implies that the bottom implication holds under K as well.

For (1), we apply structural induction to the index guard $F[\bar{i}]$. Atoms have the form $i = e$, $i \neq e$, $e \neq i$, and $i = j$, for universally quantified variables i and j and term e without universally quantified variables. When such a literal is **true** under K , then so is the corresponding literal $\text{proj}_K(i) = e$, $\text{proj}_K(i) \neq e$, $e \neq \text{proj}_K(i)$, or $\text{proj}_K(i) = \text{proj}_K(j)$, respectively, by definition of proj_K . For example, if $K \models i \neq e$, then $\alpha_K[i]$ is either equal to some $\alpha_K[j]$ for some $j \in \mathcal{I}$ such that $\alpha_K[j] \neq \alpha_K[e]$ or equal to some other value v . In the latter case, $\text{proj}_K(i) = \lambda$; that $\lambda \neq e$ is asserted in F_6 implies that $\text{proj}_K(i) \neq e$.

For the inductive case, observe that conjunction and disjunction are monotonic on truth-values: each can only become “more **true**” as its arguments switch from **false** to **true**. Thus, (1) holds.

For (2), just note that $\alpha_K[a[i]] = \alpha_K[a[\text{proj}_K(i)]]$ by the construction of J .

The bottom implication thus holds under each variant K of J , so (11.2) holds, completing the proof. $\blacksquare$

Theorem 11.8 (★Complexity). *Suppose T -satisfiability is in NP. For subfragments of the array property fragment in which formulae have bounded-size blocks of quantifiers, $(T_A \cup T)$ -satisfiability is NP-complete.*

NP-hardness follows from Theorem 9.23. That the problem is in NP follows easily from the procedure: instantiating a block of n universal quantifiers quantifying subformula G over index set $\mathcal{I}$ produces $|\mathcal{I}|^n$ new subformulae, each of length polynomial in the length of G . Hence, the output of Step 6 is of length only a polynomial factor greater than the input to the procedure for fixed n .

11.2 Integer-Indexed Arrays

Software engineers usually think of arrays as integer-indexed segments of memory. Reasoning about indices as integers provides the power of comparison via $\leq$, which enables reasoning about subarrays and properties such as that a (sub)array is sorted or partitioned. In particular, reasoning about subarrays

is essential for reasoning about programs that incrementally construct or manipulate arrays. See, for example, the programs `BubbleSort` and `QuickSort` of Chapters 5 and 6.

The **theory of integer-indexed arrays** $T_A^{\mathbb{Z}}$ augments the signature of T_A with the signature of $T_{\mathbb{Z}}$. It includes the axioms of both T_A and $T_{\mathbb{Z}}$.

11.2.1 Array Property Fragment

As in Section 11.1, we are interested in the **array property fragment** of $T_A^{\mathbb{Z}}$. An **array property** is again a $\Sigma_A^{\mathbb{Z}}$ -formula of the form

$$\forall \bar{i}. F[\bar{i}] \rightarrow G[\bar{i}] ,$$

where $\bar{i}$ is a list of integer variables, and $F[\bar{i}]$ and $G[\bar{i}]$ are the **index guard** and the **value constraint**, respectively. The form of an index guard is constrained according to the following grammar:

$$\begin{aligned} \text{iguard} &\rightarrow \text{iguard} \wedge \text{iguard} \mid \text{iguard} \vee \text{iguard} \mid \text{atom} \\ \text{atom} &\rightarrow \text{expr} \leq \text{expr} \mid \text{expr} = \text{expr} \\ \text{expr} &\rightarrow \text{uvar} \mid \text{pexpr} \\ \text{pexpr} &\rightarrow \text{pexpr}' \\ \text{pexpr}' &\rightarrow \mathbb{Z} \mid \mathbb{Z} \cdot \text{evar} \mid \text{pexpr}' + \text{pexpr}' \end{aligned}$$

where *uvar* is any universally quantified integer variable, and *evar* is any existentially quantified or free integer variable.

The form of a *value constraint* is also constrained. Any occurrence of a quantified index variable *i* must be as a read into an array, $a[i]$, for array term *a*. Array reads may not be nested; *e.g.*, $a[b[i]]$ is not allowed. Section 11.4 explains the need for these restrictions.

The array property fragment of $T_A^{\mathbb{Z}}$ then consists of formulae that are Boolean combinations of quantifier-free $\Sigma_A^{\mathbb{Z}}$ -formulae and array properties.

Example 11.9. As in the basic arrays of Section 11.1, reasoning about arrays is most useful when we can say something interesting about their elements. Suppose array elements are interpreted in some theory *T* with signature Σ . Now that both indices and elements can be interpreted in theories, we list several interesting forms of properties and their definitions for various element theories.

- Array equality $a = b$ in T_A :

$$\forall i. a[i] = b[i]$$

- Bounded array equality $\text{beq}(a, b, \ell, u)$ in $T_A^{\mathbb{Z}}$:

$$\forall i. \ell \leq i \leq u \rightarrow a[i] = b[i]$$

- Universal properties $F[x]$ in $T_A \cup T$, where $F[x]$ is a Σ -formula:

$$\forall i. F[a[i]]$$

- Bounded universal properties $F[x]$ in $T_A^{\mathbb{Z}} \cup T$, where $F[x]$ is a Σ -formula:

$$\forall i. \ell \leq i \leq u \rightarrow F[a[i]]$$

- Bounded and unbounded sorted arrays $\text{sorted}(a, \ell, u)$ in $T_A^{\mathbb{Z}} \cup T_{\mathbb{Z}}$ or $T_A^{\mathbb{Z}} \cup T_{\mathbb{Q}}$:

$$\forall i, j. \ell \leq i \leq j \leq u \rightarrow a[i] \leq a[j]$$

- Partitioned arrays $\text{partitioned}(a, \ell_1, u_1, \ell_2, u_2)$ in $T_A^{\mathbb{Z}} \cup T_{\mathbb{Z}}$ or $T_A^{\mathbb{Z}} \cup T_{\mathbb{Q}}$:

$$\forall i, j. \ell_1 \leq i \leq u_1 < \ell_2 \leq j \leq u_2 \rightarrow a[i] \leq a[j]$$

The last two predicates are necessary for reasoning about sorting algorithms, while the first four forms of properties are useful in general. For example, bounded equality is essential for summarizing the effects of a function on an array — in particular, what parts of the array are unchanged. ■

11.2.2 Decision Procedure

As in Section 11.1, the idea of the decision procedure is to reduce universal quantification to finite conjunction. Given F from the array property fragment of $T_A^{\mathbb{Z}}$, decide its $T_A^{\mathbb{Z}}$ -satisfiability as follows:

Step 1

Put F in NNF.

Step 2

Apply the following rule exhaustively to remove writes:

$$\frac{F[a \langle i \triangleleft e \rangle]}{F[a'] \wedge a'[i] = e \wedge (\forall j. j \neq i \rightarrow a[j] = a'[j])} \text{ for fresh } a' \quad (\text{write})$$

To meet the syntactic requirements on an index guard, rewrite the third conjunct as

$$\forall j. j \leq i - 1 \vee i + 1 \leq j \rightarrow a[j] = a'[j] .$$

Step 3

Apply the following rule exhaustively to remove existential quantification:

$$\frac{F[\exists \vec{i}. G[\vec{i}]]}{F[G[\vec{j}]]} \text{ for fresh } \vec{j} \quad (\text{exists})$$

Existential quantification can arise during Step 1 if the given formula has a negated array property.

Step 4

From the output of Step 3, F_3 , construct the index set $\mathcal{I}$:

$$\mathcal{I} = \{t : \cdot[t] \in F_3 \text{ such that } t \text{ is not a universally quantified variable}\} \\ \cup \{t : t \text{ occurs as a pexpr in the parsing of index guards}\}$$

If $\mathcal{I} = \emptyset$, then let $\mathcal{I} = \{0\}$. The index set contains all relevant symbolic indices that occur in F_3 .

Step 5

Apply the following rule exhaustively to remove universal quantification:

$$\frac{H[\forall \vec{i}. F[\vec{i}] \rightarrow G[\vec{i}]]}{H \left[\bigwedge_{\vec{i} \in \mathcal{I}^n} (F[\vec{i}] \rightarrow G[\vec{i}]) \right]} \quad (\text{forall})$$

n is the size of the block of universal quantifiers over $\vec{i}$.

Step 6

F_5 is quantifier-free. Decide the $(T_A \cup T_{\mathbb{Z}})$ -satisfiability of the resulting formula.

Suppose array elements are interpreted in some theory T with signature Σ . For deciding the $(T_A^{\mathbb{Z}} \cup T)$ -satisfiability of an array property $(\Sigma_A^{\mathbb{Z}} \cup \Sigma)$ -formula, use a combination decision procedure for the quantifier-free fragment of $T_A \cup T_{\mathbb{Z}} \cup T$ in Step 6. Thus, this procedure is a decision procedure precisely when the quantifier-free fragment of $T_A \cup T_{\mathbb{Z}} \cup T$ is decidable. Chapter 10 discusses deciding satisfiability in combinations of quantifier-free fragments of theories.

Example 11.10. Consider the following $\Sigma_A^{\mathbb{Z}}$ -formula:

$$F : (\forall i. \ell \leq i \leq u \rightarrow a[i] = b[i]) \\ \wedge \neg(\forall i. \ell \leq i \leq u + 1 \rightarrow a\langle u + 1 \triangleleft b[u + 1] \rangle[i] = b[i]) .$$

In NNF, we have

$$F_1 : (\forall i. \ell \leq i \leq u \rightarrow a[i] = b[i]) \\ \wedge (\exists i. \ell \leq i \leq u + 1 \wedge a\langle u + 1 \triangleleft b[u + 1] \rangle[i] \neq b[i]) .$$

Step 2 produces

$$F_2 : (\forall i. \ell \leq i \leq u \rightarrow a[i] = b[i]) \\ \wedge (\exists i. \ell \leq i \leq u + 1 \wedge a'[i] \neq b[i]) \\ \wedge a'[u + 1] = b[u + 1] \\ \wedge (\forall j. j \leq u + 1 - 1 \vee u + 1 + 1 \leq j \rightarrow a[j] = a'[j]) .$$

Step 3 removes the existential quantifier by introducing a fresh constant k :

$$F_3 : \begin{aligned} & (\forall i. \ell \leq i \leq u \rightarrow a[i] = b[i]) \\ & \wedge \ell \leq k \leq u+1 \wedge a'[k] \neq b[k] \\ & \wedge a'[u+1] = b[u+1] \\ & \wedge (\forall j. j \leq u+1-1 \vee u+1+1 \leq j \rightarrow a[j] = a'[j]) . \end{aligned}$$

Simplifying, we have

$$F'_3 : \begin{aligned} & (\forall i. \ell \leq i \leq u \rightarrow a[i] = b[i]) \\ & \wedge \ell \leq k \leq u+1 \wedge a'[k] \neq b[k] \\ & \wedge a'[u+1] = b[u+1] \\ & \wedge (\forall j. j \leq u \vee u+2 \leq j \rightarrow a[j] = a'[j]) . \end{aligned}$$

The index set is thus

$$\mathcal{I} = \{k, u+1\} \cup \{\ell, u, u+2\} ,$$

which includes the read terms k and $u+1$ and the terms ℓ , u , and $u+2$ that occur as **pexprs** in the index guards. Step 5 rewrites universal quantification to finite conjunction over this set:

$$F_5 : \begin{aligned} & \bigwedge_{i \in \mathcal{I}} (\ell \leq i \leq u \rightarrow a[i] = b[i]) \\ & \wedge \ell \leq k \leq u+1 \wedge a'[k] \neq b[k] \\ & \wedge a'[u+1] = b[u+1] \\ & \wedge \bigwedge_{j \in \mathcal{I}} (j \leq u \vee u+2 \leq j \rightarrow a[j] = a'[j]) . \end{aligned}$$

Expanding the conjunctions according to the index set $\mathcal{I}$ and simplifying according to trivially **true** or **false** antecedents ($\ell \leq u+1 \leq u$ simplifies to $\perp$, while $u \leq u \vee u+2 \leq u$ simplifies to $\top$) produces:

$$F'_5 : \begin{aligned} & (\ell \leq k \leq u \rightarrow a[k] = b[k]) \\ & \wedge (\ell \leq u \rightarrow a[\ell] = b[\ell] \wedge a[u] = b[u]) \\ & \wedge \ell \leq k \leq u+1 \wedge a'[k] \neq b[k] \\ & \wedge a'[u+1] = b[u+1] \\ & \wedge (k \leq u \vee u+2 \leq k \rightarrow a[k] = a'[k]) \\ & \wedge (\ell \leq u \vee u+2 \leq \ell \rightarrow a[\ell] = a'[\ell]) \\ & \wedge a[u] = a'[u] \wedge a[u+2] = a'[u+2] . \end{aligned}$$

$(T_A \cup T_{\mathbb{Z}})$ -satisfiability of this quantifier-free $(\Sigma_A \cup \Sigma_{\mathbb{Z}})$ -formula can be decided using the techniques of Chapter 10. But let us finish the example. F'_5 is $(T_A \cup T_{\mathbb{Z}})$ -unsatisfiable. In particular, note that by the third conjunct k is restricted such that $\ell \leq k \leq u+1$; that the first conjunct asserts that for most of this range ($k \in [\ell, u]$), $a[k] = b[k]$; that the third-to-last conjunct asserts that for $k \leq u$, $a[k] = a'[k]$, contradicting $a'[k] \neq b[k]$ by the fourth conjunct; and that even if $k = u+1$, $a'[k] \neq b[k] = b[u+1] = a'[u+1] = a'[k]$ by the fourth and fifth conjuncts, a contradiction. Hence, F is $T_A^{\mathbb{Z}}$ -unsatisfiable. ■

Theorem 11.11 (Sound & Complete). *Consider $(\Sigma_{\mathbf{A}}^{\mathbb{Z}} \cup \Sigma)$ -formula F from the array property fragment of $T_{\mathbf{A}}^{\mathbb{Z}} \cup T$. The output F_5 of Step 5 is $(T_{\mathbf{A}}^{\mathbb{Z}} \cup T)$ -equisatisfiable to F .*

The proof proceeds using the same strategy as in the proof of Theorem 11.7. The main difference is that the projection function proj_I is defined so that it maps an index to its nearest neighbor in the index set. For a given interpretation I , define a projection function $\text{proj}_I : \Sigma_{\mathbf{A}}$ -terms $\rightarrow \mathcal{I}$ that maps $\Sigma_{\mathbf{A}}^{\mathbb{Z}}$ -terms to terms of the index set $\mathcal{I}$. Let $\text{proj}_I(t) = i \in \mathcal{I}$ be such that either

- $\alpha_I[i] \leq \alpha_I[t] \wedge (\forall j \in \mathcal{I}. \alpha_I[j] \leq \alpha_I[t] \rightarrow \alpha_I[j] \leq \alpha_I[i])$
- or $\alpha_I[t] < \alpha_I[i] \wedge (\forall j \in \mathcal{I}. \alpha_I[i] \leq \alpha_I[j])$.

That is, i is the index set term that is t 's nearest neighbor under I , with preference for left neighbors. Extend proj_I to vectors of variables:

$$\text{proj}_I(\vec{i}) = (\text{proj}_I(i_1), \dots, \text{proj}_I(i_n)) .$$

Using this projection function, the remainder of the proof closely follows the proof of Theorem 11.7. Exercise 11.4 asks the reader to finish the proof.

11.3 Hashtables

Hashtables are a common data structure in modern programs. In this section, we describe a theory for hashtables $T_{\mathbf{H}}$ and provide a reduction of the hashtable property fragment of $\Sigma_{\mathbf{H}}$ -formulae into the array property fragment of $T_{\mathbf{A}}$.

The signature of $T_{\mathbf{H}}$ is the following:

$$\Sigma_{\mathbf{H}} : \{ \text{get}(\cdot, \cdot), \text{put}(\cdot, \cdot, \cdot), \text{remove}(\cdot, \cdot), \cdot \in \text{keys}(\cdot), = \} ,$$

where

- $\text{put}(h, k, v)$ is the hashtable that is modified from h by mapping key k to value v .
- $\text{remove}(h, k)$ is the hashtable that is modified from h by unmapping the key k .
- $\text{get}(h, k)$ is the value mapped by key k , which is undetermined if h does not map k to any value.
- $k \in \text{keys}(h)$ is true iff h maps the key k .

$k \in \text{keys}(h)$ is merely convenient notation for a binary predicate. However, we will exploit this notation in the following useful operations:

- **Key sets** $\text{keys}(h)$ can be unioned ($k_1 \cup k_2$), intersected ($k_1 \cap k_2$), and complemented ($\bar{k}$).
- The predicate $\text{init}(h)$ is true iff h does not map any key.

Each is definable using the basic signature.

The axioms of $T_{\mathbf{H}}$ are the following:

- $\forall x. x = x$ (reflexivity)
- $\forall x, y. x = y \rightarrow y = x$ (symmetry)
- $\forall x, y, z. x = y \wedge y = z \rightarrow x = z$ (transitivity)
- $\forall h, j, k. j = k \rightarrow \text{get}(h, j) = \text{get}(h, k)$ (hashtable congruence)
- $\forall h, j, k, v. j = k \rightarrow \text{get}(\text{put}(h, k, v), j) = v$ (read-over-put 1)
- $\forall h, k, v. \forall j \in \text{keys}(h). j \neq k \rightarrow \text{get}(\text{put}(h, k, v), j) = \text{get}(h, j)$ (read-over-put 2)
- $\forall h, k. \forall j \in \text{keys}(h). j \neq k \rightarrow \text{get}(\text{remove}(h, k), j) = \text{get}(h, j)$ (read-over-remove)
- $\forall h, k, v. k \in \text{keys}(\text{put}(h, k, v))$ (keys-put)
- $\forall h, k. k \notin \text{keys}(\text{remove}(h, k))$ (keys-remove)

Notice the similarity between the first six axioms of T_H and those of T_A . Key sets complicate the (read-over-put 2) axiom compared to the (read-over-write 2) axiom, while keys sets and key removal require three additional axioms. In particular, reading a hashtable with an unmapped key is undefined.

11.3.1 Hashtable Property Fragment

A **hashtable property** has the form

$$\forall \bar{k}. F[\bar{k}] \rightarrow G[\bar{k}] ,$$

where $F[\bar{k}]$ is the **key guard**, and $G[\bar{k}]$ is the **value constraint**. Key guards are defined exactly as index guards of the array property fragment of T_A : they are positive Boolean combinations of equalities between universally quantified keys; and equalities and disequalities between universally quantified keys k and other key terms. Value constraints can use universally quantified keys k in hashtable reads $\text{get}(h, k)$ and in key set membership checks. Finally, a hashtable property does not contain any init literals.

Σ_H -formulae that are Boolean combinations of quantifier-free Σ_H -formulae and hashtable properties comprise the **hashtable property fragment** of T_H .

Example 11.12. Consider the following hashtable property formula:

$$F : \forall k \in \text{keys}(h). \text{get}(h, k) \geq 0 .$$

Its key guard is trivial, while its value constraint is

$$k \in \text{keys}(h) \rightarrow \text{get}(h, k) \geq 0 .$$

Suppose that F annotates locations L_1 and L_2 in the following basic path:

```
@L1 : F
assume v ≥ 0;
put(h, s, v);
@L2 : F
```

We want to prove that if F holds at L_1 , then it holds at L_2 . The resulting verification condition is

$$\forall h, s, v. \left[\begin{array}{l} (\forall k \in \text{keys}(h). \text{get}(h, k) \geq 0) \wedge v \geq 0 \\ \rightarrow (\forall k \in \text{keys}(\text{put}(h, s, v)). \text{get}(\text{put}(h, s, v), k) \geq 0) \end{array} \right].$$

The key set $\text{keys}(h)$ provides a mechanism for reasoning about the incremental modification of hashtables. ■

Example 11.13. To express equality between key sets, $\text{keys}(h_1) = \text{keys}(h_2)$, in T_H , write

$$\forall k. k \in \text{keys}(h_1) \leftrightarrow k \in \text{keys}(h_2) .$$

To express equality between hashtables, $h_1 = h_2$, write

$$\text{keys}(h_1) = \text{keys}(h_2) \wedge \forall k \in \text{keys}(h_1). \text{get}(h_1, k) = \text{get}(h_2, k) .$$
■

11.3.2 Decision Procedure

Given F from the hashtable property fragment of T_H , the decision procedure reduces its T_H -satisfiability to T_A -satisfiability of a formula in the array property fragment. The main idea of the reduction is to represent hashtables h of F by two arrays in the Σ_A -formula: an array h for the elements and an array keys_h that indicates if a key maps to an element. In particular, $\text{keys}_h[k] = \circ$ indicates that k is not mapped by h , while $\text{keys}_h[k] \neq \circ$ ($\text{keys}_h[k] = \bullet$) indicates that h does map h to a value given by $h[k]$.

Step 1

Construct $F \wedge \bullet \neq \circ$, for fresh constants $\bullet$ and $\circ$.

Step 2

Rewrite F_1 according to the following set of rules:

$$\begin{aligned} F[\text{put}(h, k, v)] &\implies F[h'] \wedge h' = h \langle k \triangleleft v \rangle \wedge \text{keys}_{h'} = \text{keys}_h \langle k \triangleleft \bullet \rangle \\ F[\text{remove}(h, k)] &\implies F[h'] \wedge h = h' \langle k \triangleleft h[k] \rangle \wedge \text{keys}_{h'} = \text{keys}_h \langle k \triangleleft \circ \rangle \end{aligned}$$

for fresh variable h' . In the second rule, $h = h' \langle k \triangleleft h[k] \rangle$ expresses that position k of h' is undetermined since the mapping from k is being removed. Recall that equality $a = b$ between arrays is defined by $\forall i. a[i] = b[i]$.

Step 3

Rewrite F_2 according to the following set of rules:

$$\begin{aligned}
 F[\text{get}(h, k)] &\implies F[h[k]] \\
 F[k \in \text{keys}(h)] &\implies F[\text{keys}_h[k] \neq \circ] \\
 F[k \in K_1 \cup K_2] &\implies F[k \in K_1 \vee k \in K_2] \\
 F[k \in K_1 \cap K_2] &\implies F[k \in K_1 \wedge k \in K_2] \\
 F[k \in \overline{K}] &\implies F[\neg(k \in K)] \\
 F[\text{init}(h)] &\implies F[\forall k. \neg(k \in \text{keys}(h))]
 \end{aligned}$$

where K , K_1 , and K_2 are constructed from union, disjunction, and complementation of key set membership atoms. The final four rules define auxiliary operations: the right-side of each is expressible in the hashtable property fragment.

Step 4

Decide the T_A -satisfiability of F_3 .

As in Sections 11.1 and 11.2, hashtable values may be interpreted in some theory T with signature Σ . Then the above procedure is a decision procedure precisely when there is a decision procedure for the array property fragment of $T_A \cup T$.

Example 11.14. Consider the following Σ_H -formula:

$$G : \forall h, s, v. \left[(\forall k \in \text{keys}(h). \text{get}(h, k) \geq 0) \wedge v \geq 0 \rightarrow (\forall k \in \text{keys}(\text{put}(h, s, v)). \text{get}(\text{put}(h, s, v), k) \geq 0) \right].$$

To prove its T_H -validity, prove the T_H -unsatisfiability of the following:

$$\begin{aligned}
 F : & (\forall k \in \text{keys}(h). \text{get}(h, k) \geq 0) \wedge v \geq 0 \\
 & \wedge \neg(\forall k \in \text{keys}(\text{put}(h, s, v)). \text{get}(\text{put}(h, s, v), k) \geq 0).
 \end{aligned}$$

Step 1 introduces $\bullet$ and $\circ$:

$$F_1 : F \wedge \bullet \neq \circ.$$

Step 2 reduces hashtable modifications to arrays:

$$\begin{aligned}
 F_2 : & (\forall k \in \text{keys}(h). \text{get}(h, k) \geq 0) \wedge v \geq 0 \\
 & \wedge \neg(\forall k \in \text{keys}(h'). \text{get}(h', k) \geq 0) \\
 & \wedge h' = h \langle s \triangleleft v \rangle \wedge \text{keys}_{h'} = \text{keys}_h \langle s \triangleleft \bullet \rangle \\
 & \wedge \bullet \neq \circ.
 \end{aligned}$$

Step 3 completes the reduction:

$$\begin{aligned}
F_3 : & (\forall k. \text{keys}_h[k] \neq \circ \rightarrow h[k] \geq 0) \wedge v \geq 0 \\
& \wedge \neg(\forall k. \text{keys}_{h'}[k] \neq \circ \rightarrow h'[k] \geq 0) \\
& \wedge (\forall i. h'[i] = h\langle s \triangleleft v \rangle[i]) \wedge (\forall j. \text{keys}_{h'}[j] = \text{keys}_h\langle s \triangleleft \bullet \rangle[j]) \\
& \wedge \bullet \neq \circ .
\end{aligned}$$

F_3 is T_A -unsatisfiable. In particular, $\text{keys}_{h'} = \text{keys}_h\langle s \triangleleft \bullet \rangle$ and $h' = h\langle s \triangleleft v \rangle$, so that for all keys of h (k such that $\text{keys}_h[k] \neq \circ$), $h'[k] = h[k] \geq 0$. For the one new key s of h' , we know that $h'[s] = h\langle s \triangleleft v \rangle[s] = v \geq 0$. Therefore, no key k of h' exists at which $h[k] < 0$, a contradiction. Hence, G is T_H -valid.

The decision procedure of Section 11.1 also proves the T_A -unsatisfiability of F_3 . ■

Theorem 11.15 (Sound & Complete). *Consider $(\Sigma_H \cup \Sigma)$ -formula F from the hashtable property fragment of $T_H \cup T$. The rewriting procedure terminates, and its output F_3 of Step 3 is equisatisfiable to F and in the array property fragment of $(T_A \cup T)$.*

Proof. Inspection shows that if F is a $(\Sigma_H \cup \Sigma)$ -formula from the hashtable property fragment, then F_3 is a Σ_A -formula from the array property fragment.

The rewrite system terminates: the rules of Step 2 and the first two rules of Step 3 remove instances of $(\Sigma_H \cup \Sigma)$ -literals and $(\Sigma_H \cup \Sigma)$ -terms without introducing more; and the final four rules of Step 3 clearly terminate with a finite number of applications of the first two rules of Step 3.

We must show that F has a satisfying $(T_H \cup T)$ -interpretation precisely when F_3 has a satisfying $(T_A \cup T)$ -interpretation. Suppose that $(T_H \cup T)$ -interpretation I satisfies F . Construct the $(T_A \cup T)$ -interpretation J satisfying F_3 as follows:

- if h maps k to v in I , set $\text{keys}_h[k] = \bullet$ in J and $h[k] = v$;
- otherwise, set $\text{keys}_h[k] = \circ$ and $h[k] = v$ for some arbitrary value v .

This J satisfies the conjuncts added in Step 2; also, $h[k]$ is the same value under J as $\text{get}(h, k)$ under I , and $\text{keys}_h[k] \neq \circ$ under J iff $k \in \text{keys}(h)$ under I , showing the correctness of the first two rules of Step 3. The remaining rules are auxiliary and reduce to the first two.

Similarly, a $(T_A \cup T)$ -interpretation J satisfying F_3 corresponds to a $(T_H \cup T)$ -interpretation I satisfying F . Construct I as follows:

- if $\text{keys}_h[k] \neq \circ$ and $h[k] = v$ in J , assert $\text{get}(h, k) = v$ and $k \in \text{keys}(h)$ in I ;
- otherwise, assert $\neg(h \in \text{keys}(h))$ in I .

Again, the correspondence between the two sides of each rule is clear. ■

11.4 Larger Fragments

Why does this chapter focus on the array property fragments of T_A and $T_A^{\mathbb{Z}}$? Why not examine more expressive fragments? In fact, the following theorem

states that extending the array property fragments in natural ways produces fragments for which satisfiability is undecidable.

Theorem 11.16. *Consider the following extensions to the array property fragment of $T_A^{\mathbb{Z}}$ (T_A , where appropriate):*

- *Permit an additional quantifier alternation.*
- *Permit nested reads (e.g., $a_1[a_2[i]]$, where i is universally quantified).*
- *Permit array reads by a universally quantified variable in the index guard.*
- *Permit general Presburger arithmetic expressions over universally quantified index variables (even just addition of 1: $i + 1$) in the index guard or in the value constraint.*
- *Permit strict comparison $<$ between universally quantified variables.*
- *Augment the theory with a predicate expressing that one array is a permutation of another.*

For each resulting fragment, there exists an element theory T such that satisfiability in the array property fragment of $T_A^{\mathbb{Z}} \cup T$ ($T_A \cup T$) is decidable, yet satisfiability in the resulting fragment of $T_A^{\mathbb{Z}} \cup T$ ($T_A \cup T$) is undecidable.

Bibliographic Remarks refers the interested reader to texts that contain the proof of this theorem.

11.5 Summary

This chapter presents several decision procedures for reasoning about array-like data structures with some quantification. It covers:

- The *array property fragment* of T_A , which allows expressing properties of arrays themselves, rather than just their elements. Elements may be interpreted in some theory.
- The *array property fragment* of $T_A^{\mathbb{Z}}$, which allows expressing properties of arrays and subarrays with indices interpreted within $T_{\mathbb{Z}}$.
- Quantifier instantiation as a basis for decision procedures, which allows the direct application of decision procedures for quantifier-free fragments.
- *Hashtables*. The decision procedure rewrites a Σ_H -formula into a Σ_A -formula in which each hashtable is represented by two arrays. Reductions of this form extend decision procedures to reason about theories similar to the originally targeted theory.

Reasoning about data structures is crucial for considering the correctness of programs. The decision procedures of Chapter 9 for reasoning about recursive data structures and arrays without quantifiers provide a means of accessing elements of data structures. Additionally, equality in T_{RDS} extends to data structures. The decision procedures for the array property fragments of T_A and $T_A^{\mathbb{Z}}$ facilitate reasoning about whole or segments of arrays, not just individual elements.

Bibliographic Remarks

Theories of arrays have been studied for over four decades. McCarthy proposes the axiomatization based on read-over-write in [58]. James King implemented the decision procedure for the quantifier-free fragment as part of his thesis work [50]. Several authors discuss the quantifier-free fragment of a theory with predicates useful for reasoning about sorting algorithms [57, 43, 89]. Suzuki and Jefferson present a permutation predicate in a more restricted fragment [89]. The approximation to reasoning about the weak permutation predicate of Exercise 6.5 captures a similar fragment, though for a weaker form of permutation. Stump, Barrett, Dill, and Levitt describe a decision procedure for the quantifier-free fragment of an extensional theory of arrays [88]. Bradley, Manna, and Sipma [9] and Bradley [6] explore the array property fragment, including the proof of Theorem 11.16, that is the basis for the presentation of this chapter.

Exercises

11.1 (DP for array property fragment of T_A). Apply the decision procedure for the array property fragment of T_A to the following T_A -formulae.

- (a) $\forall i. a\langle k \triangleleft e \rangle[i] \neq e$
- (b) $a[k] = b[k] \wedge \forall i. a[i] \neq b[i]$
- (c) $a[k] \neq b[k] \wedge \forall i. a[i] = b[i]$

11.2 (DP for array property fragment of $T_A^{\mathbb{Z}}$). Apply the decision procedure for the array property fragment of $T_A^{\mathbb{Z}}$ to the following $\Sigma_A^{\mathbb{Z}}$ -formulae.

- (a) $\text{sorted}(a, \ell, u) \wedge a[\ell] > a[u]$
- (b) $\text{sorted}(a, \ell, u) \wedge e \leq a[\ell] \wedge \neg \text{sorted}(a\langle \ell - 1 \triangleleft e \rangle, \ell - 1, u)$

11.3 (DP for array property fragment of $T_A^{\mathbb{Z}}$). Apply the decision procedure for the array property fragment of $T_A^{\mathbb{Z}}$ to the following $\Sigma_A^{\mathbb{Z}}$ -formula:

$$\text{sorted}(a\langle 0 \triangleleft 7 \rangle\langle 5 \triangleleft 9 \rangle, 0, 5) \wedge \text{sorted}(a\langle 0 \triangleleft 11 \rangle\langle 5 \triangleleft 13 \rangle, 0, 5) .$$

11.4 (★Correctness of procedure for $T_A^{\mathbb{Z}}$). Prove Theorem 11.11 using the strategy suggested following the statement of the theorem.

11.5 (★Arrays with extensionality). Consider the extensional theory of arrays $T_A^=$ (see Section 3.6). Describe a decision procedure for the quantifier-free fragment of $T_A^=$ using the methods of this chapter. *Hint:* Does the index set require a fresh λ variable? Why or why not?